Lab 4: Vibe Coding

Dig Dug Repair Lab (repo: SETAPESU26/29_digDug)

Student	Atharva
SRN	PES1UG24CS093
Assigned repo	https://github.com/SETAPESU26/29_digDug
My lab repository	https://github.com/Atharva-Ashish-Vyas/SE_Lab1/tree/main/Lab-4
Before video (10 s)	https://drive.google.com/drive/folders/1DDUZGvbO1P-pkQSaY3SXo4CIIksBRK-L?usp=sharing (file: before.mp4)
After video (10 s)	https://drive.google.com/drive/folders/1DDUZGvbO1P-pkQSaY3SXo4CIIksBRK-L?usp=sharing (file: after.mp4)
Chat history link	https://claude.ai/share/556b8b58-fb6a-4fe7-a5d7-5f62e1391496

1. Summary

The assigned game is a single-file Dig Dug clone in Pygame. The README describes one deliberate bug (enemy pathfinding) and three empty extension hooks. All four tasks were completed, each as its own Git commit, and verified with an automated headless test plus before/after gameplay recordings.

#	Task	Change	Commit message
1	Fix enemy pathfinding	bfs_path: queue.pop() -> queue.popleft()	Task 1: fix BFS in bfs_path (pop -> popleft)
2	dirt_color(row)	Four distinct row-band colours	Task 2: implement dirt_color with distinct row bands
3	on_enemy_popped	18-particle burst when an enemy pops	Task 3: implement on_enemy_popped particle burst
4	enemy_speed_multiplier	Returns 1 + 0.1 * level	Task 4: implement enemy_speed_multiplier (faster per level)

2. Task 1: Fix the enemy pathfinding bug

Symptom. Enemies reached the player, but along long winding routes. **Root cause.** `bfs_path` keeps its frontier in a `deque` but removed cells with `pop()`, which takes from the right end. That makes the frontier a stack (last in, first out), so the search explores depth-first. Breadth-first search needs first-in, first-out, i.e. `popleft()`.

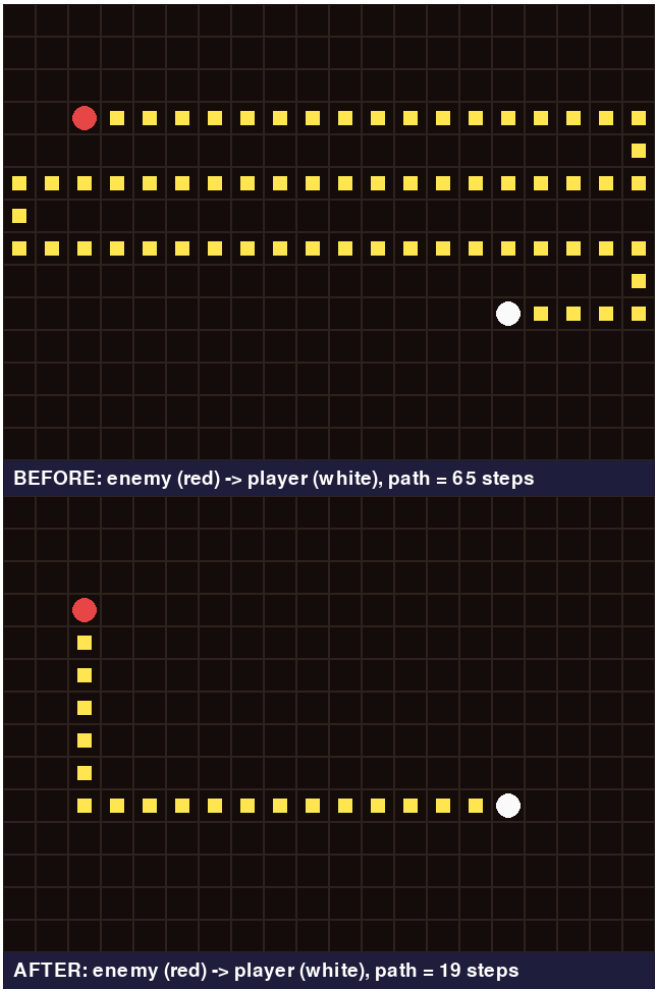
Prompt used

> *Here is my game.py. Enemies reach the player through long winding tunnels instead of the shortest path. Find the bug in bfs_path and explain why it behaves like DFS. Give me only the corrected lines.*

Diff (commit 1)

```
diff --git a/game.py b/game.py
index bad46e9..53abd85 100644
--- a/game.py
+++ b/game.py
@@ -31,7 +31,7 @@ def in_bounds(r, c):
 def bfs_path(grid, start, goal):
     queue, parents = deque([start]), {start: None}
     while queue:
-        cell = queue.pop()
+        cell = queue.popleft()
         if cell == goal:
             break
         for dr, dc in DIRS.values():
```

Evidence. Same open cavern, same start and goal. Before the fix the route is 65 steps; after it is 19 (the true shortest path, Manhattan distance 6 + 13).



3. Task 2: Implement dirt_color(row)

`draw()` calls `dirt_color(row)` once per dirt cell and falls back to the default gradient when it returns `None`. The function now returns a fixed colour per band: rows 1-3 tan topsoil, 4-6 orange clay, 7-9 reddish rock, 10-13 deep maroon. Rows outside every band return `None`, so the default gradient still applies there.

Prompt used

> *Implement dirt_color(row) in my game.py to return a distinct color per row band (4 bands across rows 1-13) instead of a smooth gradient. Return None for any row outside the bands. Don't change anything else.*

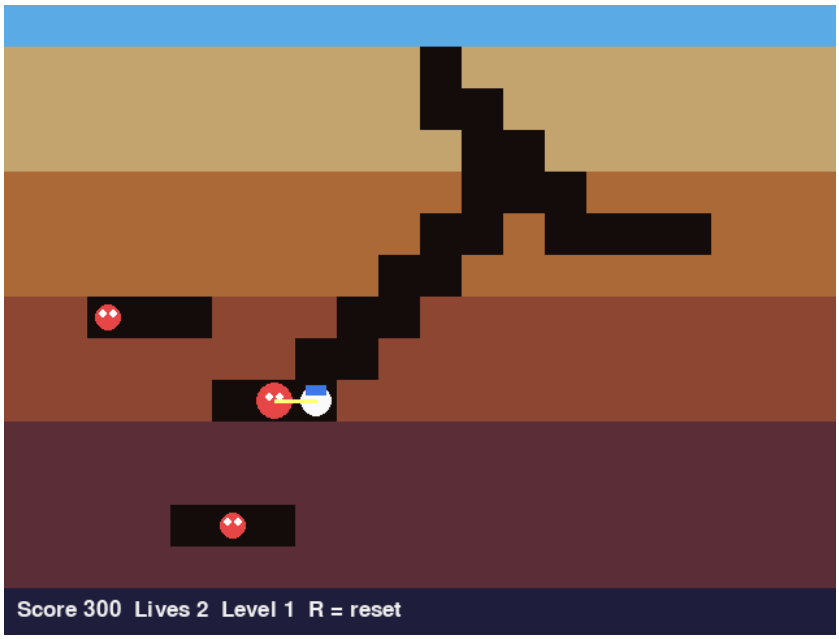
Diff (commit 2)

```
diff --git a/game.py b/game.py
index 53abd85..7dd9e91 100644
--- a/game.py
+++ b/game.py
@@ -11,7 +11,16 @@ MOVE_DELAY, ENEMY_DELAY, PUMP_RANGE, DEFLATE_AFTER = 0.11, 0.35, 3, 1.5

 def dirt_color(row):
     """Return an (r, g, b) colour for dirt in the given row, or None for the default gradient."""
-    pass
+    bands = [
+        (1, 3, (196, 164, 110)),   # topsoil
+        (4, 6, (170, 105, 55)),    # clay
+        (7, 9, (140, 70, 50)),     # reddish rock
+        (10, 13, (90, 45, 55)),    # deep maroon
+    ]
+    for lo, hi, color in bands:
+        if lo <= row <= hi:
+            return color
+    return None


 def on_enemy_popped(enemy, score):
```

Evidence: the dirt now shows four clearly separated colour bands (after screenshot).



4. Task 3: Implement on_enemy_popped(enemy, score)

The hook is called from `pump()` and its return value is ignored. Because `draw()` clears the screen every frame, the function cannot draw directly. It therefore appends particles to a module-level `PARTICLES` list. `Game.update` moves and expires them, `Game.draw` renders them, and `reset()` clears the list so none carry over to a new game.

Prompt used

> *Implement on_enemy_popped(enemy, score) as a particle burst at enemy.cell. The function is called from pump() and its return value is ignored, so use a module-level PARTICLES list and show me the minimal edits needed in Game.update and Game.draw to move and render them.*

Diff (commit 3)

```
diff --git a/game.py b/game.py
index 7dd9e91..a25a41e 100644
--- a/game.py
+++ b/game.py
@@ -9,6 +9,9 @@ DIRS = {pygame.K_UP: (-1, 0), pygame.K_DOWN: (1, 0), pygame.K_LEFT: (0, -1), pyg
 MOVE_DELAY, ENEMY_DELAY, PUMP_RANGE, DEFLATE_AFTER = 0.11, 0.35, 3, 1.5


+PARTICLES = []
+
+
 def dirt_color(row):
     """Return an (r, g, b) colour for dirt in the given row, or None for the default gradient."""
     bands = [
@@ -25,7 +28,11 @@ def dirt_color(row):

 def on_enemy_popped(enemy, score):
     """Called when an enemy is popped; add particles, bonus points, or a colour flash here."""
-    pass
+    cx = enemy.cell[1] * TILE + TILE // 2
+    cy = enemy.cell[0] * TILE + TILE // 2
+    for _ in range(18):
+        vel = pygame.math.Vector2(random.uniform(40, 160), 0).rotate(random.uniform(0, 360))
+        PARTICLES.append({"x": cx, "y": cy, "vx": vel.x, "vy": vel.y, "life": random.uniform(0.3, 0.7)})


 def enemy_speed_multiplier(level):
@@ -78,6 +85,7 @@ class Game:

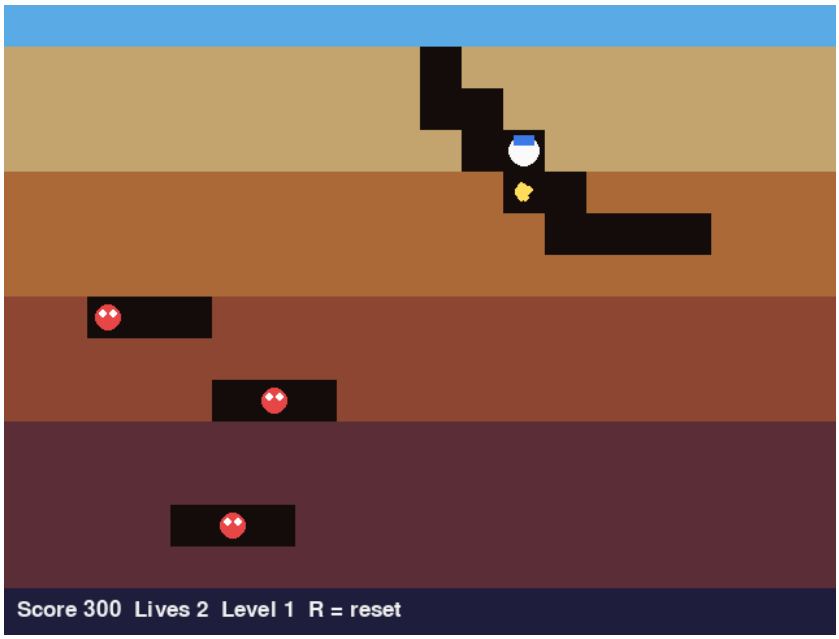
     def reset(self):
         self.level, self.score, self.lives, self.state = 1, 0, 3, "play"
+        PARTICLES.clear()
         self.start_level()

     def start_level(self):
@@ -181,6 +189,12 @@ class Game:
                 break
         if self.pump_target and (self.pump_target not in self.enemies or not self.pump_target.locked):
             self.pump_target = None
+        for p in PARTICLES[:]:
+            p["x"] += p["vx"] * dt
+            p["y"] += p["vy"] * dt
+            p["life"] -= dt
+            if p["life"] <= 0:
+                PARTICLES.remove(p)

     def draw(self, screen):
         screen.fill((30, 30, 60))
@@ -207,6 +221,8 @@ class Game:
         if self.pump_target:
             tx, ty = self.pump_target.cell[1] * TILE + TILE // 2, self.pump_target.cell[0] * TILE + TILE // 2
             pygame.draw.line(screen, (255, 255, 120), (px, py), (tx, ty), 3)
+        for p in PARTICLES:
+            pygame.draw.circle(screen, (255, 220, 90), (int(p["x"]), int(p["y"])), 3)
         hud = self.font.render(f"Score {self.score}  Lives {self.lives}  Level {self.level}  R = reset", True, (240,
```

```
        screen.blit(hud, (10, ROWS * TILE + 8))
        if self.state == "lose":
```

Evidence: yellow particles at the pop location (centre of the image, just below the player) right after an enemy is popped; score is 300.



5. Task 4: Implement enemy_speed_multiplier(level)

The step delay is `ENEMY_DELAY * (1.6 if ghost else 1) / (enemy_speed_multiplier(level) or 1)`, so a larger multiplier means a shorter delay and faster enemies. Returning `1 + 0.1 * level` gives 1.1x on level 1, 1.5x on level 5, and so on.

Prompt used

> *Implement enemy_speed_multiplier(level) so enemies get gradually faster each level. It's used as a divisor on the step delay, so a higher value means faster movement. Return 1 + 0.1 * level.*

Diff (commit 4)

```
diff --git a/game.py b/game.py
index a25a41e..9267fb6 100644
--- a/game.py
+++ b/game.py
@@ -37,7 +37,7 @@ def on_enemy_popped(enemy, score):

 def enemy_speed_multiplier(level):
     """Return a speed multiplier for enemies at the given level, or None for the default speed."""
-    pass
+    return 1 + 0.1 * level


 def in_bounds(r, c):
```

Verification

Level	Multiplier	Step delay (s), non-ghost
1	1.1x	0.318
3	1.3x	0.269
5	1.5x	0.233
10	2.0x	0.175

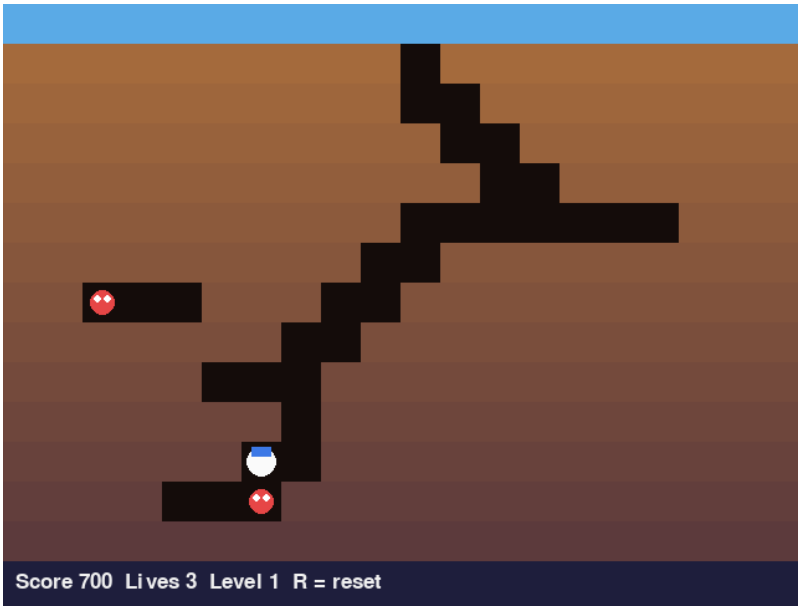
6. Automated verification

A headless pygame test (SDL dummy video driver) was run against the final code:

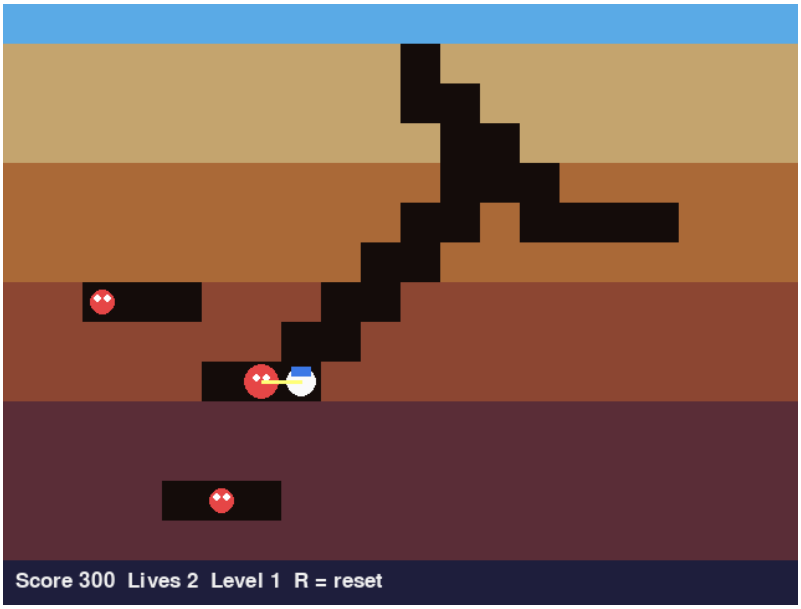
```
T1 ok: shortest paths verified (open grid 10 steps, detour 4 steps)
T2 ok: dirt_color(2)=(196,164,110), dirt_color(12)=(90,45,55), dirt_color(0)=None
T3 ok: particles 18, level 2, score 300  (4 pumps popped the enemy and advanced the level)
T3/T4 update+draw ok
Open-cavern BFS check: old=65 steps, new=19 steps
```

7. Before and after

Before (original code, bug present, plain gradient dirt, no pop effect):



After (all four tasks implemented):



Both videos are 10 seconds long:

Before video (before.mp4):
https://drive.google.com/drive/folders/1DDUZGvbO1P-pkQSaY3SXo4CIIksBRK-L?usp=sharing

After video (after.mp4):
https://drive.google.com/drive/folders/1DDUZGvbO1P-pkQSaY3SXo4CIIksBRK-L?usp=sharing

8. Prompts used (complete list)

Prompts were short and scoped to a single function so each task needed one attempt. The full conversation, including the assistant's replies, is linked below and included as the chat history deliverable.

Initial request

> *[Pasted the repo URL https://github.com/SETAPESU26/29_digDug and the full Lab 4 instructions]*

Task prompts (as listed in sections 2-5)

> *1. Here is my game.py. Enemies reach the player through long winding tunnels instead of the shortest path. Find the bug in bfs_path and explain why it behaves like DFS. Give me only the corrected lines.*
>
> *2. Implement dirt_color(row) in my game.py to return a distinct color per row band (4 bands across rows 1-13) instead of a smooth gradient. Return None for any row outside the bands. Don't change anything else.*
>
> *3. Implement on_enemy_popped(enemy, score) as a particle burst at enemy.cell. The function is called from pump() and its return value is ignored, so use a module-level PARTICLES list and show me the minimal edits needed in Game.update and Game.draw to move and render them.*
>
> *4. Implement enemy_speed_multiplier(level) so enemies get gradually faster each level. It's used as a divisor on the step delay, so a higher value means faster movement. Return 1 + 0.1 * level.*

Follow-up requests

> *"do everything for me" -> all four changes applied to a clone, tested headlessly, and delivered as four commit patches plus a push script.*
> *"now tell me step by step what to do on my github" -> GitHub repo creation and push steps.*
> *"create all these" -> this report, prompts file and submission checklist.*

Chat history link: https://claude.ai/share/556b8b58-fb6a-4fe7-a5d7-5f62e1391496

9. Commit history

```
87e11ed Task 4: implement enemy_speed_multiplier (faster per level)
3eb3472 Task 3: implement on_enemy_popped particle burst
a675ade Task 2: implement dirt_color with distinct row bands
98ec0c5 Task 1: fix BFS in bfs_path (pop -> popleft)
47a1843 edit readme for submission reqs   (original)
510d83e push dig dug                      (original)

Hashes will differ in your own repo after you push; the messages stay the same.
```

10. Reflection

The bug looked like a pathfinding failure, but the code ran and enemies did reach the player. The one-word difference between `pop()` and `popleft()` changed the search order and made the paths 3x longer in the test cavern. For the hooks, the main design point was Task 3: because the screen is cleared every frame, an effect has to live in game state that is updated and drawn each frame, not drawn inside the callback. Narrow, function-level prompts that state the contract (what is called, what is ignored) gave correct code on the first attempt.